

LLD Case Study 10: Multi-Car Elevator Control System

Target Patterns: State Pattern, Strategy Pattern, Command Pattern

Key Engineering Focus: Hardware movement states, SCAN / LOOK disk scheduling algorithms, internal/external hall call commands, and multi-elevator coordination.

1. Problem Statement & Functional Requirements

Design an automated Elevator Controller governing a bank of \$M\$ elevators across \$N\$ floors.

Requirements:

1. **Movement State Machine (State Pattern):** Elevators transition between `IDLE`, `MOVING_UP`, `MOVING_DOWN`, and `DOORS_OPEN`.
 2. **Elevator Dispatch Policy (Strategy Pattern):** Route hall calls to the most optimal car using algorithms like **LOOK / SCAN** (continue in current direction servicing all pending stops before reversing).
 3. **Command Architecture:** Requests come from external hall panels (`direction = UP/DOWN`) and internal car panels (`destination_floor`).
-

2. Architecture & State Diagram

```
stateDiagram-v2
```

```
    [*] → IDLE
```

```
    IDLE → MOVING_UP : Requests exist above
```

```
    IDLE → MOVING_DOWN : Requests exist below
```

```
    MOVING_UP → DOORS_OPEN : Arrived at scheduled floor
```

```
    DOORS_OPEN → MOVING_UP : Still have requests above
```

```
    DOORS_OPEN → MOVING_DOWN : No requests above, but have below
```

```
    DOORS_OPEN → IDLE : No pending requests
```

```
    MOVING_DOWN → DOORS_OPEN : Arrived at scheduled floor
```

```
    DOORS_OPEN → MOVING_DOWN : Still have requests below
```

```
classDiagram
    class Direction {
        <<enumeration>>
        UP
        DOWN
        IDLE
    }

    class ElevatorCar {
        +int carId
        +int currentFloor
        +Direction direction
        +Set~int~ upStops
        +Set~int~ downStops
        +step() void
        +addStop(int floor) void
    }

    class DispatchStrategy {
        <<interface>>
        +selectElevator(List~ElevatorCar~ cars, int floor, Direction dir) ElevatorCar
    }

    class ElevatorController {
        -List~ElevatorCar~ cars
        -DispatchStrategy strategy
        +requestElevator(int floor, Direction dir) void
        +selectFloor(int carId, int floor) void
    }

    ElevatorController o-- ElevatorCar : Manages
    ElevatorController o-- DispatchStrategy : Uses Strategy
```

3. Production-Grade Python Implementation

```

from enum import Enum, auto
from abc import ABC, abstractmethod
from typing import List, Set, Optional

# --- Direction & State Models ---
class Direction(Enum):
    UP = auto()
    DOWN = auto()
    IDLE = auto()

class ElevatorCar:
    def __init__(self, car_id: int):
        self.car_id = car_id
        self.current_floor = 1
        self.direction = Direction.IDLE
        self.up_stops: Set[int] = set()
        self.down_stops: Set[int] = set()

    def add_destination(self, floor: int) → None:
        if floor > self.current_floor:
            self.up_stops.add(floor)
            if self.direction == Direction.IDLE:
                self.direction = Direction.UP
        elif floor < self.current_floor:
            self.down_stops.add(floor)
            if self.direction == Direction.IDLE:
                self.direction = Direction.DOWN
        else:
            print(f"[Elevator {self.car_id}] Already at floor {floor}. Opening doors.")

    def step(self) → None:
        """LOOK (Elevator) Algorithm: Serves requests in current direction before reversing"""
        if self.direction == Direction.UP:
            if self.current_floor in self.up_stops:
                self.up_stops.remove(self.current_floor)
                print(f"[Elevator {self.car_id}] Arrived at Floor {self.current_floor}. D")

            # Check if more stops exist in UP direction
            remaining_above = [f for f in self.up_stops if f > self.current_floor]

```

```

        if remaining_above:
            self.current_floor += 1
        elif self.down_stops:
            self.direction = Direction.DOWN
        else:
            self.direction = Direction.IDLE

    elif self.direction == Direction.DOWN:
        if self.current_floor in self.down_stops:
            self.down_stops.remove(self.current_floor)
            print(f"[Elevator {self.car_id}] Arrived at Floor {self.current_floor}. D

        remaining_below = [f for f in self.down_stops if f < self.current_floor]
        if remaining_below:
            self.current_floor -= 1
        elif self.up_stops:
            self.direction = Direction.UP
        else:
            self.direction = Direction.IDLE

# --- Strategy Pattern: Elevator Dispatching ---
class DispatchStrategy(ABC):
    @abstractmethod
    def pick_car(self, cars: List[ElevatorCar], floor: int, direction: Direction) → Elev
        pass

class NearestCarStrategy(DispatchStrategy):
    """Assigns the closest car already moving in that direction or currently idle."""
    def pick_car(self, cars: List[ElevatorCar], floor: int, direction: Direction) → Elev
    def score(car: ElevatorCar) → int:
        distance = abs(car.current_floor - floor)
        if car.direction == Direction.IDLE:
            return distance
        # Penalty if car is moving away from the floor
        if car.direction == direction and (
            (direction == Direction.UP and car.current_floor ≤ floor) or
            (direction == Direction.DOWN and car.current_floor ≥ floor)
        ):
            return distance
        return distance + 100 # Heavy penalty for wrong direction

    return min(cars, key=score)

```

```

# --- Elevator Bank Controller ---
class ElevatorController:
    def __init__(self, num_cars: int):
        self.cars = [ElevatorCar(i + 1) for i in range(num_cars)]
        self.strategy = NearestCarStrategy()

    def handle_hall_call(self, floor: int, direction: Direction) → None:
        best_car = self.strategy.pick_car(self.cars, floor, direction)
        print(f"[Hall Call] Floor {floor} {direction.name} → Assigned to Elevator #{best_car.car_id}")
        best_car.add_destination(floor)

    def handle_internal_button(self, car_id: int, destination_floor: int) → None:
        car = next(c for c in self.cars if c.car_id == car_id)
        car.add_destination(destination_floor)

    def simulate_step(self) → None:
        for car in self.cars:
            if car.direction != Direction.IDLE:
                car.step()

# --- Verification Driver ---
if __name__ == "__main__":
    controller = ElevatorController(num_cars=2)

    # User at floor 5 presses UP
    controller.handle_hall_call(floor=5, direction=Direction.UP)

    # Passenger inside Elevator 1 presses 8
    controller.handle_internal_button(car_id=1, destination_floor=8)

    print("\n--- Simulating 6 Elevator Clock Cycles ---")
    for cycle in range(6):
        print(f"\n[Tick {cycle + 1}]")
        controller.simulate_step()

```